

# manvr3d: A Platform for Human-in-the-loop Cell Tracking in Virtual Reality

Samuel Pantze\*  
CASUS, Görlitz, Germany  
Helmholtz-Zentrum  
Dresden-Rossendorf e.V.  
Dresden, Germany

Jean-Yves Tinevez†  
Institut Pasteur  
Université Paris Cité  
Image Analysis Hub (IAH)  
75015 Paris, France

Matthew McGinity‡  
IXLAB, Technische  
Universität Dresden,  
Germany

Ulrik Günther§  
Helmholtz-Zentrum  
Dresden-Rossendorf e.V.  
Dresden, Germany

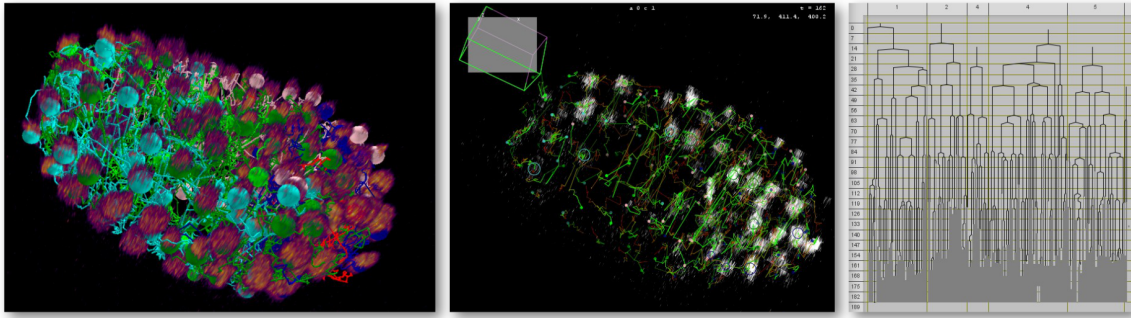


Figure 1: Cell tracking in Mastodon (middle), 3D visualization of volume and track data in sciview (left) and a cell lineage tree (right). The dataset shows the early development stage of a *C. elegans* roundworm. Downloaded from the Cell Tracking Challenge website [15], data courtesy of Waterston Lab, University of Washington, Seattle, WA, USA [17].

## ABSTRACT

We propose *manvr3d*, a VR platform for immersive, AI-assisted human-in-the-loop cell tracking. Life scientists reconstruct the developmental history of organisms at the cellular level by analyzing 3D time-lapse microscopy images acquired at high spatio-temporal resolution. However, reconstruction of cell trajectories and lineage trees is a highly time consuming and error prone task. Common tools are often limited to 2D image display, which greatly limits spatial understanding and navigation. Deep Learning-based algorithms accelerate this process, yet depend heavily on manually-annotated, high-quality ground truth data and curation. In this work, we bridge the gap between Deep Learning-based cell tracking software and 3D/VR visualization to create a hybrid AI-human-in-the-loop cell tracking system. We lift the incremental annotation, training and proofreading loop of the deep learning model into the third dimension and apply natural user interfaces like hand gestures and eye tracking to accelerate the cell tracking workflow for life scientists. We present here the technical architecture of our platform and first analysis of performance. Our code is released open source.

**Index Terms:** Systems Biology, Virtual Reality, Microscopy, Cell Tracking, Volume Rendering, Eye Tracking.

## 1 INTRODUCTION

Modern microscopes enable biologists to capture large scale 3D time-lapse datasets of embryonic development and other multi-cellular structures. Tracking of the imaged cells over time is a vital—yet non-trivial—task in the workflow of a life scientist studying the function and development of cells, tissues, and organisms. The result of the

tracking process is a cell lineage tree (see Fig. 1, right) that encodes information about the cellular ancestry.

The tracking process consists classically of two stages. In the first *detection* stage, cells are located in individual images or image stacks. In some cases, this might also include segmentation, in which cell shapes and boundaries are extracted in addition to cell positions. In the second *linking* step, individual cells are matched between successive images, allowing cell trajectories and lineage to be extracted. Tracking algorithms perform these tasks automatically on the input image. This scientific topic has received considerable attention and many tools are available today. Recent algorithms rely either on conventional image processing, statistical methods (like Gaussian mixture models [2]) or deep learning methods [16]. Evaluating their performance or training supervised deep learning models require ground truth annotations, which are extremely time-consuming to create due to the effort involved in manually annotating cells and their links across frames.

One solution to this problem is to use sparse annotations combined with incremental human-in-the-loop deep learning. The neural cell-tracking model learns from human feedback, provided in the form of iterative cycles of corrections to the model’s predictions. This is the approach taken by ELEPHANT [26]. However, with ELEPHANT, the human is constrained to a traditional mouse and keyboard interface and 2D display of 2D slices of the 3D data. This interface is not only slow but also error-prone, where the researcher might miss crucial spatial context.

In this work, we present *manvr3d* (Multimodal ANnotations in Virtual Reality 3D, pronounced “*manfred*”), extending the approach taken by ELEPHANT by bringing the annotation and linking steps into virtual reality (VR) and enabling users to perform these tasks with VR controllers and eye tracking. The central contributions of this work are:

1. *manvr3d*, a bridge between a widely-used, open-source cell tracking software and a 3D rendering engine, enabling bidirectional editing capabilities between 2D and 3D components, serving as a platform for developing natural user interface-based cell tracking solutions, and

\*e-mail: s.pantze@hzdr.de

†e-mail: tinevez@pasteur.fr

‡e-mail: matthew.mcginity@tu-dresden.de

§e-mail: ulrik.guenther@hzdr.de

- Two implementations of VR-based cell tracking—One using handheld controllers in an interactive VR environment, and another using eye-tracking hardware in VR to accelerate the tracking process even further.

In this paper we describe the development of a functioning prototype. A full user study is beyond the scope of this work, in [Sec. 5](#) however, we show indicative numbers for both rendering and annotation performance.

## 2 RELATED WORK

Various software solutions exist for analyzing and visualizing biological data in 3D and/or VR environments. For cell lineage data, visualization and annotation solutions exist for both 2D [[13](#), [22](#)] and 3D [[9](#), [23](#)]. Neither of these support immersive rendering in VR. Leeuw et al. [[3](#)] address this gap by rendering cell trajectories in a CAVE<sup>1</sup> system with superimposed volume time-series. None of those solutions integrate the visualization aspects into a wider analysis platform and do not offer integration of a machine learning model.

Additionally, proprietary solutions like *ConfocalVR* [[25](#)] or *syGlass* [[18](#)] offer general quantification and measurement tools for biological image data, *Arivis Pro VR*<sup>2</sup> also offers features for cell tracking in volume data in VR. Arivis and syGlass have been utilized by Kaltenecker et al. [[12](#)] to annotate cell segmentations, which they then use to train their 3D deep learning model *DELiVR* for automated cell segmentation to speed up the annotation process. Note that in this work, no cell tracking is performed. Elor et al. developed the mixed reality environment *BioLumin* [[4](#)] to study the efficacy of crowd-sourced tissue annotation tasks, where the collected data is later used for training deep learning models. VR has been used in [[5](#)] to segment biological 3D time-series data, [[28](#)] visualized meshes resulting from the segmentation process, and [[21](#)] further annotate those meshes in VR. Again, no cell tracking is performed.

In contrast to the aforementioned solutions, we present an immersive open-source visualization platform for human-in-the-loop cell tracking with natural user interfaces to enable a streamlined and ergonomic process for both the creation of ground truth data and the final proofreading step. Our system is embedded into the Fiji [[24](#)] ecosystem, resulting in easier adoption and integration into existing pipelines.

## 3 IMPLEMENTATION

We first detail the software ecosystem, then explain our platform together with the data structures used. Finally, we will describe both our handheld controller-based tracking solution, as well as the eye tracking-based one.

### 3.1 Software ecosystem

*manvr3d* integrates with Fiji/ImageJ [[24](#)], a widely-used software package for both visualization and analysis of scientific and biological image data. For the purpose of this project, we rely on three existing Fiji plugins and frameworks:

- Mastodon* [[20](#)] as a platform for cell tracking, supporting the annotation of very large datasets. Mastodon uses *Big-DataViewer* [[19](#)] as a backend to efficiently render large volumetric data as 2D slices. Tracking is performed manually using mouse and keyboard interaction, or semi-automatically with a difference-of-Gaussians filter for cell detection and either a Kalman filter or LAP linker [[11](#)] for cell linking.
- ELEPHANT* [[26](#)] extends Mastodon with an incremental deep learning model. The model is first trained on a sparse dataset,

consisting only of a handful of manual annotations. In a proof-reading loop, the user then corrects the predictions and iteratively trains the model again and again, quickly increasing the size of the training dataset. A second (optional) U-Net model is trained on optical flow prediction, using the existing cell links as training data, to guide the linking of spots between time points.

- Sciview* [[6](#)] and its underlying rendering framework *scenery* [[8](#)], to allow visualization of large volumetric data together with 3D meshes. Both tools support a variety of natural user interfaces, such as VR headsets, or eye tracking hardware.

By extending widely-used open-source software packages, we aim to broaden the appeal of our platform to users, and encourage researchers to extend upon it further. Within Mastodon, *manvr3d* acts as an extension and can be opened by selecting Window > New *manvr3d* view.

### 3.2 Software platform

*manvr3d* is essentially a bridge that facilitates the interplay between Mastodon/ELEPHANT and sciview, enabling the reconstruction of a cell lineage tree in 3D, superimposed with a volume rendering of the image dataset. *manvr3d* allows for bidirectional editing of the track data using VR controllers and other NUI devices. The same training and prediction commands that are available in the regular 2D interface of ELEPHANT are also available from within the VR environment.

### 3.3 Organization and Data structure

*manvr3d* orchestrates all connections between the different components and ensures data consistency between the 2D data representation in Mastodon and the 3D representation in sciview. The data flow is visualized in [Fig. 2](#) with color coded components. *manvr3d* (green) handles track editing events bidirectionally from either side and updates the other side accordingly. Time point changes are also communicated across components to maintain synchronized 2D (blue) and 3D (orange) views. Detailed information about Mastodons internal data structure are provided in [Supplement A.1](#). We offer a graphical user interface that allows adjustment of various visualization parameters as well as launching a VR session (purple), either with or without additional hardware, such as eye trackers. The functionality of ELEPHANT (turquoise) is integrated as a menu inside the VR environment, enabling access to the training and prediction steps for the incremental deep learning model.

Annotated cell positions (spots) are only rendered for the current time point. Cell trajectories on the other hand span a longer time range, and as such they are treated independently of the spots. The bridge maintains a hash map of all links in the scene, which allows us to rapidly toggle the visibility of each segment to create a sliding window effect when moving through time. Using the time point information stored in the hash map, it is also possible to color each track with color maps that range from the minimum to the maximum time point. The effect of track coloring can be seen in [Fig. 3](#). Individual spot editing events, like additions, movement and deletions, are handled on a per-spot and per-segment basis by event handlers and so do not cause a full graph redraw. To that end, the bridge locks update requests from event listeners on either side if an update is already in progress to prevent feedback loops and inconsistencies.

## 4 CELL TRACKING WITH NATURAL USER INTERFACES

We provide two implementations for tracking cells in VR with natural user interfaces: using controllers (and in the future possibly hand gestures) and an experimental implementation for utilizing eye tracking hardware, where the user's gaze directions are analyzed and cell tracks are created from this information. For intuitive interaction

<sup>1</sup>Cave automatic virtual environment

<sup>2</sup>[zeiss.com/microscopy/en/products/software/arivis-pro.html](https://zeiss.com/microscopy/en/products/software/arivis-pro.html)

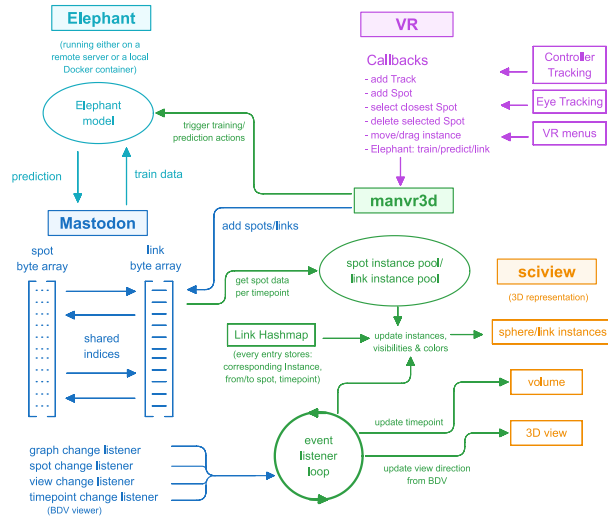


Figure 2: Data layout for *manvr3d*. It maintains a 3D spot and track representation in sciview via an event listener loop. Editing events are handed back to Mastodon, where they trigger partial or full graph redraw events (see appendix for details).

with the VR environment, we drew inspiration from popular VR productivity and creativity software like Gravity Sketch<sup>3</sup> or Shapelab<sup>4</sup>. Common one- and two-handed gestures for moving the observer, as well as scaling, rotating and translating of the dataset are implemented. A detailed controller layout can be found in Supplement A.2. Clicking a spot highlights it, and allows the user to either move it to a new position or delete it. In the same manner, an arbitrary number of new spots can be placed to annotate all cells in the current time point.

ELEPHANT actions are coupled to buttons on a wrist menu, allowing the user to interact with the deep learning model from within the VR environment. We currently support buttons for assigning the true positive label to all spots in the scene, for triggering the training, prediction and linking actions.

These interaction methods act as a basis for both controller-based and eye tracking-based cell tracking.

#### 4.1 Cell Tracking with Handheld Controllers

A 3D cursor in form of a small sphere is attached to the right VR controller and allows interaction with the VR environment. Cell position annotation is performed by moving the 3D cursor into the target cell and pressing the right trigger button. Through the semi-transparent rendering of the image data, it is possible to precisely position the cursor inside the desired cell. With each annotation click, time is automatically advanced, making the tracing of a cell very rapid, by repeatedly clicking into the cell. By default, time advances backwards, as this is found to simplify handling of cell division events. This process is repeated until either the first time point is reached or the user manually terminates the tracking process by pressing the right B button.

The recorded track is sent to Mastodon only after tracking is finished to prevent continuous redrawing of the 3D representation. After the data are included in Mastodon's graph data structure, a full redraw event is triggered and the 3D tracks are updated.

It is possible to merge an active track into an existing spot—a cell division, since the animation is played backwards—by simply

<sup>3</sup>gravitysketch.com

<sup>4</sup>shapelabvr.com

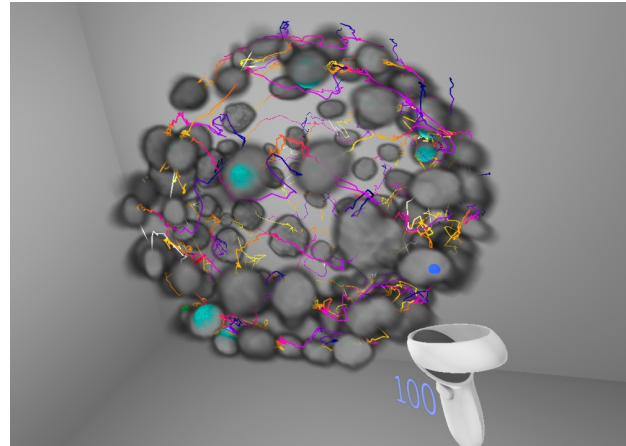


Figure 3: VR user looking at a *Platynereis* dataset, annotated with controllers and ELEPHANT. Data courtesy of Tomancak Lab, MPI-CBG.

clicking on the target spot. The opposite is also possible: Clicking an existing spot with the trigger button to start a new track will use that spot as its origin point. Using both of these features in conjunction thus allows for bridging holes in existing tracks.

#### 4.2 Cell Tracking via Eye Tracking

Tracking cells using gaze interaction has been explored in *Bionic Tracking* [7] by Günther et al. We incorporate this technique into our platform. While following a moving cell through the 3D volume data with their gaze, we record the user's gaze directions and sample the volume at a uniform interval along each gaze ray. Finding the position of a cell along this gaze ray thus turns into a 1-dimensional problem, because we can assume that the first local maximum along the ray corresponds to the target cell. After calculating the local maxima for each ray, the track is reconstructed with a variant of the A\* algorithm [27] that connects the closest local maxima from subsequent gaze rays, see Fig. 4.

To avoid the Midas touch problem [10], it is important to remove any visual distractions that could lead the user to unintentionally look away from their target cell. For this reason, we implemented a dual input approach for cell tracking by eye tracking: as soon as the user is ready to start tracking a cell with their eyes, they press the left trigger button. This starts playback of the dataset and the continuous collection of gaze directions and volume density samples along each ray. Once the user interrupts the tracking with the trigger button, or if the first time point is reached, the collected gaze rays and their sampled values are analyzed and subsequently sent to Mastodon. A collection of rays is plotted in Fig. 4, where each ray originates from the positive Y axis and extends downwards. Changes over time are plotted along the X axis.

### 5 PERFORMANCE AND RESULTS

#### 5.1 Rendering Performance

In *manvr3d*, we reuse the same underlying data allocated by the BigDataViewer backend in Mastodon. Especially with large datasets, this reduces memory load significantly, compared to solutions where memory sharing is not possible and copies are necessary.

Both primitive types of the 3D representation, spheres for spots and cylinders for links, are currently rendered as instanced meshes on the GPU. Two instance pools are populated during the initialization phase—one for spots and one for links. We found that instance pre-generation is faster than on-the-fly generation. The instances in these pools are then positioned and colored according to the current

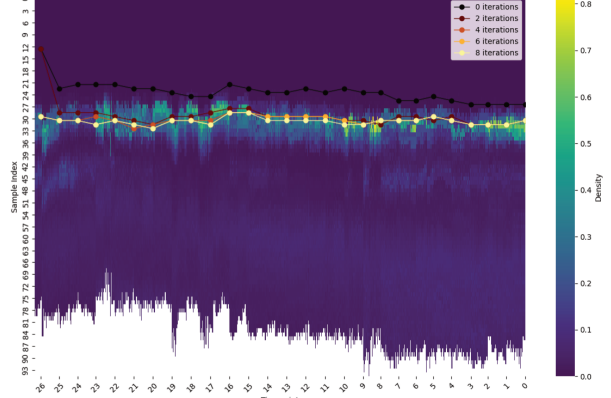
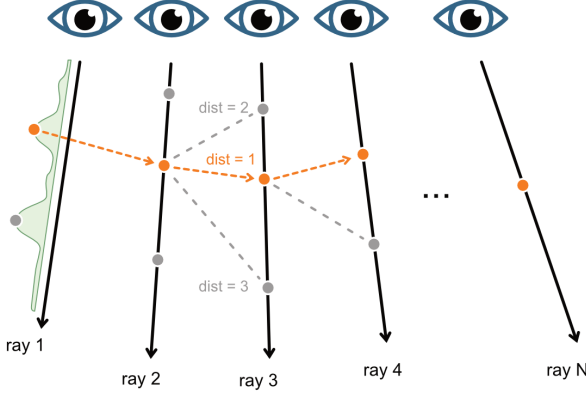


Figure 4: *Left*: Scheme of the graph search algorithm that connects the closest local maxima in subsequent gaze rays following the first local maximum found in the first ray. *Right*: A collection of gaze rays across time, collected by following a moving cell with one’s eyes. We slide a simple Gauss kernel in the shape of  $[0.25, 0.5, 0.25]$  along each ray to smooth the signal and extract local maxima more effectively. Here we show the effect of various iterations of Gauss smoothing. It can be seen that no or low amounts of smoothing can lead to incorrectly extracted tracks, and starting from 4 iterations onward we are able to extract the correct cell track.

time point. If a time point requires more spots, they are dynamically allocated to the pool.

We benchmarked *manvr3d*’s capability to render geometry instances (see Tab. 1) with two differently-sized datasets and found that for up to several ten thousand cells, there is no negative effect on performance<sup>5</sup>. Turning off the overlaid volume rendering yields slightly faster frame rates.

Table 1: Frame rate and scene population time, depending on the number of geometry instances rendered in the scene. The scene population time is only required once during scene initialization.

| Dataset size                    | Small  | Large     |
|---------------------------------|--------|-----------|
| Number of links                 | 3000   | 243,000   |
| Spots rendered per time point   | 90-110 | 2500-3700 |
| Frame rate with volume (fps)    | 185    | 29        |
| Frame rate without volume (fps) | 230    | 32        |
| Scene population time (s)       | 0.2    | 15        |

## 5.2 Annotation performance

As stated before, performing a full user study is beyond the scope of this work. We instead performed two partial annotations (Tab. 2) of a *Platynereis* dataset and a *Drosophila* dataset [14], to compare annotation speeds between the Mastodon approach (2D), VR-based controller tracking (VR) and ELEPHANT (+DL). We measured the time taken to create 10 tracks each. We then used the pre-trained *versatile* ELEPHANT model and trained it over 5 epochs on the annotations created, and divided the final amount of tracks (104 for *Platynereis* and 74 for *Drosophila*) by the sum of training and prediction time. The training/prediction time is independent of the manual annotation method used.

These numbers indicate that VR-based tracking can significantly outperform manual 2D annotations, by a factor of about 6. Although the time needed for prediction and training in the ELEPHANT model remains the same for both methods, *manvr3d* can provide a significant benefit for the annotation aspect.

<sup>5</sup>The frame rates refer to a render resolution of 1920x1080 pixels, on an XMG Fusion 15 laptop with NVIDIA RTX 4070 GPU and Intel Core i9-14900HX processor, running Windows 10 build 19044.5371, JDK version 21.0.5, NVIDIA driver version 566.36.

Table 2: Annotation times for 2D, VR and Deep Learning tracking.

| Dataset            | Size                         | Type | Time/track      |
|--------------------|------------------------------|------|-----------------|
| <i>Platynereis</i> | 700x660x113, 101 time points | 2D   | 3.85 min        |
|                    |                              | VR   | <b>0.65 min</b> |
|                    |                              | +DL  | 0.125 min       |
| <i>Drosophila</i>  | 151x101x29, 31 time points   | 2D   | 1.07 min        |
|                    |                              | VR   | <b>0.16 min</b> |
|                    |                              | +DL  | 0.02 min        |

## 6 SUMMARY AND FUTURE WORK

In this work, we presented *manvr3d*, a platform to allow VR/natural user interface-based cell tracking together with two example implementations, using handheld controllers or eye tracking hardware. We couple this annotation process with the ELEPHANT deep learning model for rapid training data acquisition and provide track editing tools for proofreading of the predictions. *manvr3d* easily scales to several thousand cells.

The ELEPHANT model provides uncertainty information to indicate the confidence of the network’s prediction. We plan to incorporate these data into the visualization process by coloring the spots and tracks accordingly, thus guiding the user towards areas of potentially higher error rate. Visualizing these uncertainty data is still an area of active research [1].

To quantify the improvement—indicated in Sec. 5.2—of our human-in-the-loop tracking approach over conventional methods, we plan to conduct a user study that compares the speed and accuracy of fully manual methods with automated methods and the approach taken in this project, using a variety of datasets.

## 7 SOFTWARE AVAILABILITY

The software can be found in the Github repository at [github.com/scenerygraphics/manvr3d](https://github.com/scenerygraphics/manvr3d). A fully-packaged version for easy deployment on Windows systems can be downloaded at [github.com/scenerygraphics/manvr3d/releases/](https://github.com/scenerygraphics/manvr3d/releases/).

## ACKNOWLEDGMENTS

The authors thank Ko Sugawara for his ongoing support with the ELEPHANT integration. Thanks to Vladimir Ulman and Ruoshan Lan for their work on a prototype of this project, and to Jan Tiemann

for providing code and support for 3D user interface elements. The authors also want to thank Elias Barriga, Valentin Ruffine, Ana Patricia Ramos and Jaime Hidalgo for their feedback on the software.

SP was partially funded by the Center for Advanced Systems Understanding (CASUS), financed by Germany's Federal Ministry for Research, Technology, and Space (BMFTR) and by the Saxon Ministry for Science, Culture and Tourism (SMWK) with tax funds on the basis of the budget approved by the Saxon State Parliament. JYT was funded by the French National Research Agency (France BioImaging, ANR-24-INBS-0005 FBI BIOGEN). UG was supported by *GoBio Initial*, 03LW0622, by the Germany's Federal Ministry for Research, Technology, and Space (BMFTR).

## REFERENCES

- [1] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, V. Makarenkov, and S. Nahavandi. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 76:243–297, 2021. doi: 10.1016/j.inffus.2021.05.008 4
- [2] F. Amat, W. Lemon, D. P. Mossing, K. McDole, Y. Wan, K. Branson, E. W. Myers, and P. J. Keller. Fast, accurate reconstruction of cell lineages from large-scale fluorescence microscopy data. *Nature Methods*, 11(9), 2014. doi: 10.1038/nmeth.3036 1
- [3] W. De Leeuw, R. Van Liere, P. Verschure, A. Visser, E. Manders, and R. Van Drielf. Visualization of time dependent confocal microscopy data. In *Proceedings Visualization 2000. VIS 2000 (Cat. No.00CH37145)*, pp. 473–476, Oct. 2000. doi: 10.1109/VISUAL2000.885735 2
- [4] A. Elor, S. Whittaker, S. Kurniawan, and S. Michael. BioLumin: An Immersive Mixed Reality Experience for Interactive Microscopic Visualization and Biomedical Research Annotation. *ACM Transactions on Computing for Healthcare*, 3(4):44:1–44:28, Nov. 2022. doi: 10.1145/3548777 2
- [5] C. Guérinot, V. Marcon, C. Godard, T. Blanc, H. Verdier, G. Planchon, F. Raimondi, N. Boddart, M. Alonso, K. Sailor, P.-M. Lledo, B. Hajji, M. El Beheiry, and J.-B. Masson. New Approach to Accelerated Image Annotation by Leveraging Virtual Reality and Cloud Computing. *Frontiers in Bioinformatics*, 1, 2022. doi: 10.3389/fbinf.2021.777101 2
- [6] U. Günther and K. I. S. Harrington. Tales from the Trenches: Developing sciview, a new 3D viewer for the ImageJ community. In C. Gillmann, M. Krone, G. Reina, and T. Wischgoll, eds., *VisGap - The Gap between Visualization Research and Visualization Software*. The Eurographics Association, 2020. doi: 10.2312/visgap.20201112 2
- [7] U. Günther, K. I. S. Harrington, R. Dachselt, and I. F. Sbalzarini. Bionic Tracking: Using Eye Tracking to Track Biological Cells in Virtual Reality. In A. Bartoli and A. Fusiello, eds., *Computer Vision – ECCV 2020 Workshops*, pp. 280–297. Springer International Publishing, Cham, 2020. doi: 10.1007/978-3-030-66415-2\_18 3
- [8] U. Günther, T. Pietzsch, A. Gupta, K. I. Harrington, P. Tomancak, S. Gumhold, and I. F. Sbalzarini. scenery: Flexible virtual reality visualization on the java vm. In *2019 IEEE Visualization Conference (VIS)*, pp. 1–5, 2019. doi: 10.1109/VISUAL2019.8933605 2
- [9] J. Hong, A. Trubuil, and T. Isenberg. LineageD: An Interactive Visual System for Plant Cell Lineage Assignments based on Correctable Machine Learning. *Computer Graphics Forum*, 41(3):195–207, 2022. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14533>. doi: 10.1111/cgf.14533 2
- [10] R. J. K. Jacob. Eye Tracking in Advanced Interface Design. In *Virtual Environments and Advanced Interface Design*. Oxford University Press, July 1995. doi: 10.1093/oso/9780195075557.003.0015 3
- [11] K. Jaqaman, D. Loerke, M. Mettlen, H. Kuwata, S. Grinstein, S. L. Schmid, and G. Danuser. Robust single-particle tracking in live-cell time-lapse sequences. *Nature Methods*, 5(8):695–702, Aug. 2008. Number: 8 Publisher: Nature Publishing Group. doi: 10.1038/nmeth.1237 2
- [12] D. Kaltenecker, R. Al-Maskari, M. Negwer, L. Hoehner, F. Kofler, S. Zhao, M. Todorov, Z. Rong, J. C. Paetzold, B. Wiestler, M. Piraud, D. Rueckert, J. Geppert, P. Morigny, M. Rohm, B. H. Menze, S. Herzig, M. Berriel Diaz, and A. Ertürk. Virtual reality-empowered deep-learning analysis of brain cells. *Nature Methods*, pp. 1–10, Apr. 2024. Publisher: Nature Publishing Group. doi: 10.1038/s41592-024-02245-2 2
- [13] D. Lange, R. Judson-Torres, T. A. Zangle, and A. Lex. Aardvark: Composite Visualizations of Trees, Time-Series, and Images. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):1290–1300, Jan. 2025. doi: 10.1109/TVCG.2024.3456193 2
- [14] W. Lemon. Drosophila embryo tissue time-lapse., July 2019. doi: 10.5281/zenodo.3336346 4
- [15] M. Maška, V. Ulman, P. Delgado-Rodriguez, E. Gómez-de Mariscal, T. Nečasová, F. A. Guerrero Peña, T. I. Ren, E. M. Meyerowitz, T. Scherr, K. Löffler, R. Mikut, T. Guo, Y. Wang, J. P. Allebach, R. Bao, N. M. Al-Shakarji, G. Rahmon, I. E. Toubal, K. Palaniappan, F. Lux, P. Matula, K. Sugawara, K. E. G. Magnusson, L. Aho, A. R. Cohen, A. Arbel, T. Ben-Haim, T. R. Raviv, F. Isensee, P. F. Jäger, K. H. Maier-Hein, Y. Zhu, C. Ederra, A. Urbila, E. Meijering, A. Cunha, A. Muñoz-Barrutia, M. Kozubek, and C. Ortiz-de Solórzano. The Cell Tracking Challenge: 10 years of objective benchmarking. *Nature Methods*, 20(7):1010–1020, July 2023. Publisher: Nature Publishing Group. doi: 10.1038/s41592-023-01879-y 1
- [16] E. Moen, D. Bannon, T. Kudo, W. Graf, M. Covert, and D. Van Valen. Deep learning for cellular image analysis. *Nature Methods*, 16(12):1233–1246, Dec. 2019. doi: 10.1038/s41592-019-0403-1 1
- [17] J. I. Murray, Z. Bao, T. J. Boyle, M. E. Boeck, B. L. Mericle, T. J. Nicholas, Z. Zhao, M. J. Sandel, and R. H. Waterston. Automated analysis of embryonic gene expression with cellular resolution in *C. elegans*. *Nature Methods*, 5(8):703–709, Aug. 2008. doi: 10.1038/nmeth.1228 1
- [18] S. Pidhorskyi, M. Morehead, Q. Jones, G. Spirou, and G. Doretto. syGlass: Interactive Exploration of Multidimensional Images Using Virtual Reality Head-mounted Displays, Aug. 2018. arXiv:1804.08197 [cs]. doi: 10.48550/arXiv.1804.08197 2
- [19] T. Pietzsch, S. Saalfeld, S. Preibisch, and P. Tomancak. BigDataViewer: visualization and processing for large image data sets. *Nature Methods*, 12(6):481–483, June 2015. doi: 10.1038/nmeth.3392 2
- [20] T. Pietzsch, J.-Y. Tinevez, M. Arzt, V. Ulman, K. Sugawara, and S. Hahmann. Mastodon. <https://mastodon.readthedocs.io/en/latest/>. 2, 6
- [21] A. Platt, E. J. Lutton, E. Offord, and T. Bretschneider. MiCellAnn-GELo: annotate microscopy time series of complex cell surfaces with 3D virtual reality. *Bioinformatics*, 39(1):btad013, Jan. 2023. doi: 10.1093/bioinformatics/btad013 2
- [22] A. J. Pretorius, I. A. Khan, and R. J. Errington. Cell lineage visualisation. *Computer Graphics Forum*, 34(3):21–30, June 2015. doi: 10.1111/cgf.12614 2
- [23] I. Salvador-Martínez, M. Grillo, M. Averof, and M. Telford. CeLaVi: an interactive cell lineage visualization tool. *Nucleic Acids Research*, 49(W1):W80–W85, May 2021. doi: 10.1093/nar/gkab325 2
- [24] J. Schindelin, I. Arganda-Carreras, E. Frise, V. Kaynig, M. Longair, T. Pietzsch, S. Preibisch, C. Rueden, S. Saalfeld, B. Schmid, J.-Y. Tinevez, D. White, V. Hartenstein, K. Eliceiri, P. Tomancak, and A. Cardona. Fiji: An open-source platform for biological-image analysis. *Nature methods*, 9:676–82, 06 2012. doi: 10.1038/nmeth.2019 2
- [25] C. Stefani, A. Lacy-Hulbert, and T. Skillman. ConfocalVR: Immersive Visualization for Confocal Microscopy. *Journal of Molecular Biology*, 430(21):4028–4035, Oct. 2018. doi: 10.1016/j.jmb.2018.06.035 2
- [26] K. Sugawara, Ç. Çevrim, and M. Averof. Tracking cell lineages in 3d by incremental deep learning. *eLife*, 2022. doi: 10.7554/eLife.69380 1, 2
- [27] X. Sun, W. Yeoh, and S. Koenig. Dynamic fringe-saving A\*. In *Proceedings of The 8th International Conference on Autonomous Agents and Multiagent Systems - Volume 2, AAMAS '09*, pp. 891–898. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, May 2009. doi: 10.5555/1558109.1558136 3
- [28] X. Zhang, M. Almasian, S. Hassan, R. Jotheesh, V. Kadam, A. Polk, A. Saberigarani, A. Rahat, J. Yuan, J. Lee, K. Carroll, and Y. Ding. 4D Light-sheet imaging and interactive analysis of cardiac contractility in zebrafish larvae. *APL Bioengineering*, 7(2), 2023. doi: 10.1063/5.0153214 2